

Cronograma de Actividades y Entregables

Proyecto: Robot Autónomo - Módulo de Ecuaciones Diferenciales

Duración: 4 Semanas

Responsable:	Equipo de Proyecto Integrador
Asignatura:	Ecuaciones Diferenciales
Docente:	Msc. Víctor Rodríguez Estévez
Fecha de inicio:	12 de mayo de 2026
Fecha de entrega final:	8 de junio de 2026
Integra con:	Física III, Microcontroladores, Diseño 3D, Metodología de la Investigación, Estadística II, C#

Índice

1. Resumen Ejecutivo del Cronograma	3
2. Cronograma Detallado Semana a Semana	4
2.1. Semana 1: Modelado del Motor DC y Ecuaciones Diferenciales	4
2.2. Semana 2: Transformada de Laplace y Control PID	5
2.3. Semana 3: Métodos Numéricos y Código para Microcontrolador	6
2.4. Semana 4: No Linealidades, Linealización y Entrega Final	7
3. Diagrama de Gantt del Proyecto	9
4. Entregables Detallados	10
4.1. Entregables por Semana	10
4.2. Estructura de la Documentación Final	11
5. Integración con Otras Asignaturas - ¿Qué entregar a cada una?	12
6. Recursos y Herramientas	13
6.1. Recursos de Hardware	13
6.2. Stack Tecnológico	13
7. Riesgos y Plan de Mitigación	14
8. Checklist de Cumplimiento por Semana	15
8.1. Semana 1 Checklist	15
8.2. Semana 2 Checklist	15
8.3. Semana 3 Checklist	15
8.4. Semana 4 Checklist	16
8.5. Checklist General del Proyecto	16
9. Firmas y Aprobaciones	17

1. Resumen Ejecutivo del Cronograma

El presente documento establece el cronograma detallado para el desarrollo del componente de **Ecuaciones Diferenciales** del robot autónomo. Incluye modelado matemático del motor DC, aplicación de Transformada de Laplace para control PID, implementación de métodos numéricos (RK4), análisis de no linealidades y generación de entregables para las otras 6 asignaturas del proyecto integrador.

El proyecto tiene una duración total de **4 semanas (20 días hábiles)**, con entregas parciales al final de cada semana y una entrega final consolidada.

Resumen del cronograma por semana

Cuadro 1: Resumen por semana

Semana	Tema Central	Herramienta temática	Ma-	Entregable Principal
Semana 1	Modelado y EDO	Solución analítica + Euler		Modelo matemático documentado
Semana 2	Laplace y PID	Transformada de Laplace + Ziegler-Nichols		Función de transferencia + PID simulado
Semana 3	Métodos Numéricos	Runge-Kutta 4 orden		Código C# para microcontrolador
Semana 4	No Linealidades + Integración	Linealización + Phase Portrait		Notebook final + presentación

Distribución de esfuerzo estimada

- Modelado y EDO: **20 %** del tiempo total
- Laplace y control PID: **25 %** del tiempo total
- Métodos numéricos y porteo a C#: **25 %** del tiempo total
- No linealidades e integración final: **30 %** del tiempo total

2. Cronograma Detallado Semana a Semana

2.1. Semana 1: Modelado del Motor DC y Ecuaciones Diferenciales

Objetivo: Derivar las ecuaciones eléctrica y mecánica del motor DC, escribir el sistema en espacio de estados, implementar solución analítica y método de Euler.

Cuadro 2: Actividades Semana 1 (Días 1-5)

Día	Actividad	Responsable/Tecnología	Horas
Día 1	Identificación de parámetros físicos del motor (R, L, Kt, Ke, J, B)	Todo el equipo	4
Día 1	Derivación de ecuación eléctrica (Kirchhoff + Faraday)	Coordinador	4
Día 2	Derivación de ecuación mecánica (Newton para rotación)	Coordinador	4
Día 2	Escritura del sistema en forma de espacio de estados	Todo el equipo	4
Día 3	Implementación de solución analítica en Python	Programador	4
Día 3	Implementación de método de Euler (verificación)	Programador	4
Día 4	Comparación de soluciones analítica vs numérica	Programador	4
Día 4	Validación de errores (tolerancia $< 10^{-3}$)	Todo el equipo	4
Día 5	Preparación de informe Semana 1	Documentador	4
Día 5	Revisión y checklist	Todo el equipo	4

Entregable Semana 1

- Informe técnico con ecuaciones derivadas y justificación de parámetros
- Notebook de Python con solución analítica y método de Euler
- Gráfica comparativa (solución exacta vs Euler)
- Checklist de cumplimiento firmado

Métricas de éxito

- Error máximo de Euler $< 5\%$ respecto a solución analítica
- Todas las ecuaciones correctamente escritas en LaTeX
- Código ejecutable y documentado

2.2. Semana 2: Transformada de Laplace y Control PID

Objetivo: Aplicar Laplace a la EDO linealizada, derivar función de transferencia, sintonizar controladores PID con Ziegler-Nichols y simular lazo cerrado.

Cuadro 3: Actividades Semana 2 (Días 6-10)

Día	Actividad	Responsable/Tecnología	Horas
Día 6	Linealización de la EDO alrededor de ω_0	Coordinador	4
Día 6	Aplicación de Transformada de Laplace	Coordinador	4
Día 7	Derivación de función de transferencia $G(s) = \frac{\Omega(s)}{V(s)}$	Coordinador	4
Día 7	Cálculo de ganancia última K_u y período último T_u	Todo el equipo	4
Día 8	Sintonización PID con tabla Ziegler-Nichols	Todo el equipo	4
Día 8	Implementación de simulación lazo cerrado en Python	Programador	4
Día 9	Simulación de controladores P, PI, PID	Programador	4
Día 9	Comparación de overshoot, tiempo de subida, e_{ss}	Todo el equipo	4
Día 10	Documentación de ecuaciones de Laplace	Documentador	4
Día 10	Preparación de informe Semana 2	Documentador	4

Entregable Semana 2

- Función de transferencia $G(s)$ derivada
- Tabla de parámetros PID (K_p , K_i , K_d) calculados
- Simulación en Python de lazo cerrado con P, PI, PID
- Gráficas comparativas de respuesta ante escalón
- Informe con análisis de estabilidad

Métricas de éxito

- Error en estado estacionario $e_{ss} = 0$ para controlador PI/PID
- Overshoot $< 20\%$ para PID sintonizado
- Tiempo de subida $< 0,3$ segundos

2.3. Semana 3: Métodos Numéricos y Código para Microcontrolador

Objetivo: Implementar Runge-Kutta de orden 4 desde cero, comparar errores con Euler y traducir el código a C# para flashear en el microcontrolador.

Cuadro 4: Actividades Semana 3 (Días 11-15)

Día	Actividad	Responsable/Tecnología	Horas
Día 11	Derivación de ecuaciones RK4 para EDO del motor	Todos	4
Día 11	Implementación de RK4 desde cero (sin scipy) en Python	Programador	4
Día 12	Comparación de errores: Euler vs RK4	Programador	4
Día 12	Gráfica logarítmica de errores vs Δt	Programador	4
Día 13	Determinación de Δt óptimo (precisión vs velocidad)	Ingeniero firmware	4
Día 13	Traducción de RK4 a C# / C++ para microcontrolador	Ingeniero firmware	4
Día 14	Implementación de PID discreto en C#	Ingeniero firmware	4
Día 14	Pruebas de estabilidad numérica con diferentes Δt	Todos	4
Día 15	Simulación de condiciones adversas (ruido, saturación)	Programador	4
Día 15	Preparación de informe Semana 3 + código documentado	Documentador	4

Entregable Semana 3

- Código RK4 en Python (desde cero, sin librerías de alto nivel)
- Código en C# para microcontrolador (Arduino/STM32)
- Gráfica comparativa: error RK4 vs Euler (escala log-log)
- Tabla de Δt óptimo recomendado
- Informe con análisis de convergencia

Métricas de éxito

- Error RK4 $< 10^{-6}$ comparado con solución exacta
- Código C# compila sin errores
- PID discreto funcionando en simulación

2.4. Semana 4: No Linealidades, Linealización y Entrega Final

Objetivo: Identificar y modelar no linealidades (fricción de Coulomb, saturación, zona muerta), linealizar alrededor de punto de operación, generar campo de direcciones e integrar todos los módulos.

Cuadro 5: Actividades Semana 4 (Días 16-20)

Día	Actividad	Responsable/Tecnología	Horas
Día 16	Identificación de no linealidades del sistema real	Todos	4
Día 16	Modelado matemático de fricción de Coulomb	Coordinador	4
Día 17	Modelado de saturación del actuador y zona muerta	Ingeniero firmware	4
Día 17	Linealización del sistema alrededor de ω_0	Coordinador	4
Día 18	Cálculo de matriz Jacobiana y valores propios	Todos	4
Día 18	Generación de campo de direcciones (phase portrait)	Programador	4
Día 19	Simulación del sistema no lineal completo	Programador	4
Día 19	Comparación simulación lineal vs no lineal	Todos	4
Día 20	Integración de notebook final de 4 semanas	Programador	4
Día 20	Preparación de presentación final (10-15 min)	Todos	4

Entregable Semana 4 (Final)

- Notebook final de Python con todos los módulos (ETL, CNN, LSTM/GRU, Transformer) – aunque esto es del otro proyecto, aquí corresponde: modelo no lineal + linealización + PID
- Código C# final para microcontrolador con no linealidades incluidas
- Campo de direcciones (phase portrait) del sistema
- Informe final consolidado (mínimo 20 páginas)
- Presentación ejecutiva (10-15 diapositivas)
- Demo funcional del sistema simulado

Métricas de éxito

- Valores propios del sistema linealizado indican estabilidad
- Phase portrait coherente con la física del motor
- Diferencia entre modelo lineal y no lineal $< 15\%$ en rango operativo

- Todos los checklists completados

3. Diagrama de Gantt del Proyecto

Cuadro 6: Diagrama de Gantt - Semana 1

Actividad	D1	D2	D3	D4	D5
Identificación de parámetros					
Derivación ecuaciones eléctrica/-mecánica					
Espacio de estados					
Solución analítica + Euler					
Validación de errores					
Informe Semana 1					
E1: Entregable					

Cuadro 7: Diagrama de Gantt - Semana 2

Actividad	D6	D7	D8	D9	D10
Linealización + Laplace					
Función de transferencia					
Ziegler-Nichols (Ku, Tu)					
Simulación P, PI, PID					
Análisis comparativo					
Informe Semana 2					
E2: Entregable					

Leyenda

- □ Actividades de modelado y EDO
- □ Actividades de Laplace y PID
- □ Actividades de métodos numéricos (RK4)
- □ Actividades de no linealidades e integración
- □ Entregables (E1, E2, E3, E4)

Cuadro 8: Diagrama de Gantt - Semana 3

Actividad	D11	D12	D13	D14	D15
Implementación RK4					
Comparación Euler vs RK4					
Porteo a C#					
PID discreto					
Pruebas estabilidad					
Informe Semana 3					
E3: Entregable					

Cuadro 9: Diagrama de Gantt - Semana 4

Actividad	D16	D17	D18	D19	D20
Modelado no linealidades					
Linealización + Jacobiano					
Phase portrait					
Simulación no lineal					
Integración final + notebook					
Presentación final					
E4: Entregable Final					

4. Entregables Detallados

4.1. Entregables por Semana

Semana	Entregable	Formato	Fecha
Semana 1	Modelo matemático documentado	PDF + Notebook (.ipynb)	Fin día 5
Semana 1	Solución analítica + Euler	Código Python	Fin día 5
Semana 1	Checklist Semana 1 firmado	PDF	Fin día 5
Semana 2	Función de transferencia $G(s)$	PDF	Fin día 10
Semana 2	Tabla parámetros PID	PDF + código	Fin día 10
Semana 2	Simulación lazo cerrado	Notebook + gráficas	Fin día 10
Semana 3	RK4 desde cero en Python	Código + notebook	Fin día 15
Semana 3	Código C# para microcontrolador	Archivo .cs	Fin día 15
Semana 3	Gráfica comparativa errores	Imagen + análisis	Fin día 15

Semana 4	Modelo no lineal completo	Código + notebook	Fin día 20
Semana 4	Phase portrait	Imagen + análisis	Fin día 20
Semana 4	Informe final consolidado	PDF (20+ páginas)	Fin día 20
Semana 4	Presentación ejecutiva	PPT/PDF (10-15 slides)	Fin día 20
Semana 4	Demo funcional	Video/en vivo	Fin día 20

4.2. Estructura de la Documentación Final

La documentación final deberá incluir los siguientes capítulos:

1. Resumen ejecutivo (1-2 páginas)
2. Introducción y objetivos del proyecto
3. Modelado matemático del motor DC (EDO, espacio de estados)
4. Aplicación de Laplace y diseño de controladores PID
5. Implementación de métodos numéricos (Euler, RK4, comparación)
6. No linealidades: identificación, modelado y linealización
7. Análisis de estabilidad (valores propios, phase portrait)
8. Integración con otras asignaturas (6 materias)
9. Resultados experimentales y métricas
10. Conclusiones y trabajo futuro
11. Apéndices (código completo, parámetros, checklists)

5. Integración con Otras Asignaturas - ¿Qué entregar a cada una?

Este cronograma debe presentarse a las otras 6 asignaturas para coordinar entregables y fechas.

Asignatura	Entregable desde Ecuaciones Diferenciales	Fecha de entrega sugerida
Física III	Tabla comparativa K_e teórico vs medido. Gráfica voltaje inducido vs ω	Fin Semana 2
Microcontroladores	Código C# con RK4 y PID discreto listo para flashear	Fin Semana 3
Diseño 3D	Parámetros geométricos: radio de rueda r , distancia entre ruedas L . Ecuaciones cinemáticas derivadas	Fin Semana 1
Metodología de la Investigación	Pregunta de investigación: ".Efecto de fricción de Coulomb en tiempo de establecimiento". Hipótesis formulada. Diseño experimental	Fin Semana 2
Estadística II	Datos de velocidad simulada (50 repeticiones con ruido). Intervalos de confianza para K , τ , K_p , K_i , K_d	Fin Semana 4
Programación C# (POO)	Clase <code>MotorDC</code> con métodos <code>RK4Step()</code> , <code>PIDUpdate()</code> . Herencia, interfaces, eventos	Fin Semana 3

Flujo de comunicación inter-asignaturas

Ecuaciones Diferenciales (modelo matemático, PID, RK4, C#)		
↓		
→ Física III:	K_e experimental	(valida leyes electromagnetismo)
→ Microcontroladores:	Código C# listo para flashear	(control en tiempo real)
→ Diseño 3D:	Parámetros r , L , cinemática	(diseño mecánico)
→ Metodología:	Hipótesis + diseño experimental	(método científico)
→ Estadística II:	Datos + intervalos de confianza	(análisis cuantitativo)
→ Programación C#:	Clase <code>MotorDC</code> + RK4 + PID	(POO aplicada)

6. Recursos y Herramientas

6.1. Recursos de Hardware

Cuadro 12: Requerimientos de hardware

Componente	Mínimo	Recomendado
CPU	2 núcleos	4+ núcleos
RAM	8 GB	16 GB
GPU	No necesaria (opcional)	Para simulaciones grandes
Almacenamiento	10 GB SSD	20 GB SSD
Microcontrolador	Arduino Uno	STM32 o ESP32

6.2. Stack Tecnológico

- **Modelado y simulación:** Python 3.9+ con NumPy, SciPy, Matplotlib
- **Cálculo simbólico:** SymPy (para Laplace y EDO)
- **Código para microcontrolador:** C# / C++ (Arduino IDE, PlatformIO)
- **Documentación:** $\text{\LaTeX}$ + Overleaf
- **Control de versiones:** Git + GitHub/GitLab
- **Notebooks:** Jupyter Notebook / Google Colab

7. Riesgos y Plan de Mitigación

Riesgo	Impacto	Mitigación
Parámetros del motor no disponibles	Retraso en modelado	Usar parámetros de referencia del Anexo B
Código C# no compila en microcontrolador	No se puede probar en hardware	Probar en simulador antes de flashear
RK4 mal implementado (errores grandes)	Simulaciones incorrectas	Validar con solución analítica conocida
No linealidades ignoradas hasta el final	Modelo poco realista	Incorporar fricción desde Semana 3
Poca coordinación con otras asignaturas	Entregables fuera de tiempo	Reunión semanal de integración
Tiempo insuficiente en Semana 4	Entregas incompletas	Priorizar phase portrait y linealización

8. Checklist de Cumplimiento por Semana

8.1. Semana 1 Checklist

- ☐ Identificar y documentar parámetros físicos del motor (R , L , K_t , K_e , J , B)
- ☐ Derivar ecuación eléctrica (Ley de Kirchhoff + Faraday)
- ☐ Derivar ecuación mecánica (Segunda ley de Newton para rotación)
- ☐ Escribir sistema en forma de espacio de estados
- ☐ Implementar solución analítica en Python
- ☐ Implementar método de Euler como verificación
- ☐ Comparar soluciones analítica y numérica (gráfica superpuesta)
- ☐ Validar que error Euler $<$ tolerancia
- ☐ Preparar informe Semana 1

8.2. Semana 2 Checklist

- ☐ Linealizar EDO alrededor de ω_0
- ☐ Aplicar Transformada de Laplace a EDO linealizada
- ☐ Derivar función de transferencia $G(s)$
- ☐ Calcular parámetros Ziegler-Nichols (K_u , T_u)
- ☐ Determinar K_p , K_i , K_d con tabla de sintonización
- ☐ Implementar simulación de lazo cerrado en Python
- ☐ Simular controlador P y calcular error estacionario
- ☐ Simular controlador PI y verificar $e_{ss} = 0$
- ☐ Simular controlador PID completo
- ☐ Comparar tiempos de subida y overshoot entre P, PI, PID
- ☐ Documentar ecuaciones de Laplace en informe

8.3. Semana 3 Checklist

- ☐ Derivar ecuaciones RK4 para EDO del motor
- ☐ Implementar RK4 desde cero (sin scipy) en Python
- ☐ Comparar error RK4 vs Euler (gráfica logarítmica)
- ☐ Determinar Δt óptimo (precisión vs velocidad)
- ☐ Traducir RK4 a C# / C++ para microcontrolador

- ☐ Implementar PID discreto en C#
- ☐ Probar estabilidad numérica con diferentes Δt
- ☐ Simular condiciones adversas (ruido, saturación)
- ☐ Código C# compila y está documentado
- ☐ Preparar informe Semana 3

8.4. Semana 4 Checklist

- ☐ Identificar no linealidades del sistema real
- ☐ Modelar fricción de Coulomb matemáticamente
- ☐ Modelar saturación del actuador
- ☐ Modelar zona muerta (dead zone)
- ☐ Linealizar sistema alrededor de ω_0
- ☐ Calcular matriz Jacobiana y valores propios
- ☐ Analizar estabilidad del sistema linealizado
- ☐ Generar campo de direcciones (phase portrait)
- ☐ Simular sistema no lineal completo
- ☐ Comparar simulación lineal vs no lineal
- ☐ Integrar notebook final de 4 semanas
- ☐ Preparar presentación final (10-15 min)
- ☐ Escribir conclusiones y recomendaciones

8.5. Checklist General del Proyecto

- ☐ ¿Todos los miembros entienden el modelo matemático?
- ☐ ¿RK4 tiene error $< 10^{-6}$ comparado con solución exacta?
- ☐ ¿La simulación lineal coincide con la analítica?
- ☐ ¿El PID reduce overshoot y tiempo de subida?
- ☐ ¿Los valores propios indican estabilidad?
- ☐ ¿El código C# está probado en simulación?
- ☐ ¿Se identificaron al menos 3 no linealidades?
- ☐ ¿El campo de direcciones tiene sentido físico?
- ☐ ¿Se entregan todos los informes semanales?
- ☐ ¿La presentación está ensayada?
- ☐ ¿Se cumplió el cronograma de 4 semanas?

9. Firmas y Aprobaciones

Rol	Firma
Responsable de Ecuaciones Diferenciales	_____
Fecha:	_____
Docente (Msc. Víctor Rodríguez Estévez)	_____
Fecha:	_____
Representante de Física III	_____
Representante de Microcontroladores	_____
Representante de Diseño 3D	_____
Representante de Metodología	_____
Representante de Estadística II	_____
Representante de Programación C#	_____

Nota final: El presente cronograma es un documento vivo y puede ser ajustado durante la ejecución del proyecto previa justificación y aprobación de los responsables. Cualquier desviación superior a 2 días en una actividad debe ser reportada inmediatamente al docente y a los representantes de las otras asignaturas.